

Continuous AI Model Alignment for Smart Manufacturing

via Digital Twin Constraint Propagation and Multi-Criteria Open-Source Model Selection

Kevin Omede¹, Alessandro Simeone¹, and Yi Li²

¹Department of Management and Production Engineering, Politecnico di Torino, Turin, Italy, {kevin.omede, alessandro.simeone}@polito.it

²School of Automation Science and Electrical Engineering, Beihang University, Beijing, China, liyi@buaa.edu.cn

Abstract

Predictive maintenance and process monitoring in smart manufacturing depend on AI models that remain accurate as production conditions evolve. In practice, equipment ageing, batch variation, and process changeovers continuously alter the statistical distribution of sensor signals, degrading any model calibrated at deployment time. At the same time, the heterogeneous edge hardware found on the shop floor—ranging from embedded systems such as NVIDIA Jetson Nano to industrial PCs—imposes strict latency and memory budgets that disqualify many high-accuracy models regardless of their benchmark performance. No existing framework addresses both challenges within a single closed-loop pipeline that is governed by the formal asset context encoded in a factory Digital Twin (DT).

This paper presents **DMCA** (Dynamic Model-Context Alignment), a five-stage framework that keeps the deployed AI model continuously aligned with both the current

hardware profile of the production cell and the current statistical regime of its sensors. Stage 1 ingests the Asset Administration Shell (AAS) descriptor of the target edge device, extracting hardware and service-level agreement (SLA) constraints. Stage 2 applies a TOPSIS multi-criteria ranking over a curated catalogue of open-source time-series foundation models hosted on HuggingFace, weighting accuracy ($w = 0.5$), inference latency ($w = 0.3$), parameter count ($w = 0.1$), and license permissivity ($w = 0.1$). Stage 3 enforces a four-check quality gate before committing any model to production. Stage 4 runs a two-layer concept drift detector that classifies seven distinct drift types with 92 % held-out accuracy and zero false alarm rate, enabling type-specific re-alignment strategies. Stage 5 orchestrates shadow deployment and routes swap decisions through a large language model (LLM) copilot that provides conversational explanations to non-technical production operators.

Experiments on eight time-series foundation models across NASA CMAPSS FD001 (predictive maintenance) and ETT-h1 (energy forecasting) benchmarks demonstrate that the highest-accuracy model ($\text{MAE} = 0.006$) is inadmissible on Jetson Nano hardware due to a 2,039 ms inference latency against a 50 ms SLA, while the TOPSIS-selected model ($\text{MAE} = 0.019$, latency = 5.86 ms) satisfies all constraints with full sensor-coverage. On the PRONOSTIA bearing dataset, DMCA's admitted model delivers a 430 s warning lead time ahead of bearing failure—exceeding the 300 s operator-action threshold—compared with 280 s for a higher-accuracy but latency-constrained alternative. Drift detection on an ETT-h1 concept-drift scenario achieves a 20-timestep end-to-end recovery cycle, reducing post-drift MAE from 3.121 to 0.174.

Keywords: smart manufacturing, digital twin, asset administration shell, predictive maintenance, model selection, TOPSIS, concept drift, time-series forecasting, edge AI, conversational explainability, open-source AI, Industry 4.0

1 Introduction

Modern smart manufacturing cells generate continuous streams of vibration, temperature, current, and pressure signals from instrumented machinery. The economic case for AI-driven

predictive maintenance is well established: unplanned downtime in automotive and aerospace production can cost between \$22,000 and \$50,000 per hour [1], and Overall Equipment Effectiveness (OEE) targets above 85 % require alarm lead times that exceed the time needed to source replacement parts and schedule maintenance personnel [2]. Yet AI adoption on the shop floor remains fragmented, for two structural reasons.

The hardware-constraint problem. Production cells are equipped with edge devices whose compute budgets are constrained by power envelopes, certification requirements, and cost. An NVIDIA Jetson Nano (5 W, ARM Cortex-A57) is representative of a broad class of embedded inference hosts deployed in manufacturing. The growing ecosystem of open-source time-series foundation models hosted on public repositories such as HuggingFace offers attractive accuracy, but their inference latencies span three orders of magnitude — from under 6 ms to over 2,000 ms — and no systematic selection mechanism exists that maps these models onto hardware profiles described by the factory Digital Twin.

The non-stationarity problem. Industrial processes are inherently non-stationary. Tool wear, raw-material batch variation, seasonal temperature gradients, and scheduled maintenance events all alter the distribution of sensor signals over time, causing any fixed model to degrade gradually or abruptly. Statistical process control (SPC) charts detect such shifts for univariate signals, but they do not trigger the model replacement, re-validation, and operator notification required to maintain predictive-maintenance coverage.

Existing frameworks address either problem in isolation. Hardware-aware neural architecture search [8] optimises model design for edge budgets but does not incorporate the structured asset context of the factory Digital Twin, nor does it consider pre-trained open-source models. Concept-drift detectors such as ADWIN [3] signal distribution change with theoretical false-alarm guarantees but leave the subsequent model-selection and operator-communication steps to manual intervention. The Digital Twin literature has converged on the Asset Administration Shell (AAS) as the standardised format for encoding hardware, sensor, and SLA constraints [1, 2], but existing AAS deployments are used for monitoring and simulation, not as live inputs to AI orchestration.

We propose **DMCA** (Dynamic Model-Context Alignment), a closed-loop framework that

treats the Digital Twin as a continuously updated constraint source and re-aligns the deployed model whenever the AAS profile or the sensor distribution changes. DMCA is structured as a five-stage pipeline that spans data ingestion, model admission, quality gating, drift detection, and re-alignment with operator-facing conversational explainability.

The original contributions of this paper are:

- C1. DT-driven closed-loop model orchestration for manufacturing.** The first framework that uses AAS-encoded hardware and SLA constraints as live inputs to a multi-criteria model ranking, re-triggering selection whenever the Digital Twin is updated (Section 3, Stage 1–2).
- C2. TOPSIS-based admission of open-source time-series models under edge SLA constraints.** A four-criterion TOPSIS formulation—accuracy, inference latency, parameter count, and license—validated against eight HuggingFace foundation models on two industrial datasets (Section 4).
- C3. Seven-type concept drift classifier with zero false alarm rate.** A two-layer detector combining ADWIN with an eight-feature decision-tree classifier that distinguishes abrupt, gradual, incremental, variance, distribution, outlier-driven, and unclassified drift, achieving 92 % held-out accuracy and FAR = 0.000 (Section 3, Stage 4).
- C4. Safety-aware alarm-coverage metric for bearing failure validation.** A monitoring-coverage formula derived from the scheduling model that quantifies whether a model's inference cadence is fast enough to preserve actionable alarm lead time under a given SLA (Section 4).

The remainder of this paper is organised as follows. Section 2 reviews related work across Digital Twins, time-series foundation models, multi-criteria selection, and concept drift in manufacturing. Section 3 describes the five DMCA stages in detail. Section 4 presents experimental results on CMAPSS, ETT-h1, and PRONOSTIA. Section 5 discusses limitations and industrial deployment considerations. Section 6 concludes with directions for future work.

2 Related Work

2.1 Digital Twins and Asset Administration Shells in Manufacturing

The concept of the Digital Twin — a continuously updated virtual replica of a physical asset — was formalised by Grieves and Vickers and has since been operationalised across aerospace, energy, and discrete manufacturing [10]. In the Industry 4.0 context, the *Asset Administration Shell* (AAS) provides the standardised, machine-readable representation of a DT's properties. The IDTA specification defines two AAS types: Type 1 (static file, serialised in JSON or XML) for documentation and offline exchange, and Type 2 (reactive API) for live integration into control loops. The IEC 63278-1:2023 standard [2] formalises the submodel concept, enabling typed descriptions of hardware, sensor configuration, and service-level agreements alongside process data.

Rahal *et al.* [9] demonstrate AAS integration with predictive-maintenance pipelines for rotating machinery, showing that structured asset metadata improves alarm prioritisation. Xia *et al.* (Stuttgart) push this further by using large language models to *generate* AAS documentation from technical PDFs, reducing the authoring burden for industrial operators. However, neither work uses the AAS as a live input to a model-selection pipeline: the AI model deployed on the asset is still chosen manually or fixed at commissioning time.

The gap addressed here is structural: no published framework couples AAS submodel parsing to an automated, continuously re-triggered model-ranking step that enforces the asset's current hardware and SLA constraints as hard admissibility criteria before any accuracy-based comparison.

2.2 Open-Source Time-Series Foundation Models for Manufacturing

A new class of open-source time-series foundation models has emerged on public model repositories, offering zero-shot and few-shot generalisation to unseen domains without dataset-specific training. PatchTST [4] demonstrates that patch-based transformer encoders transfer effectively across diverse time-series tasks; Moirai [5] introduces a unified multi-dataset training regime that produces strong zero-shot forecasters; and Lag-Llama [6] adapts the

LLaMA decoder architecture for probabilistic time-series prediction. Chronos (Amazon) further extends this paradigm by framing forecasting as a language-modelling task over quantised value tokens.

Despite their accuracy on server-grade benchmarks, these models pose significant challenges for edge deployment in smart manufacturing. Their inference latencies span a 348:1 range, from 3 ms to 2,039 ms on a single NVIDIA T4 GPU [3]. When scaled to embedded controllers such as the Jetson Nano (a commonly deployed 5 W inference host in manufacturing cells), these latencies imply monitoring coverage rates as low as 9 %, meaning the model processes only 1 in 11 sensor windows — a critical gap that existing model-card documentation does not report.

Surveys of AI in manufacturing confirm that practitioners typically select models based on task-accuracy benchmarks alone, ignoring hardware constraints that determine whether the model can actually sustain the required alarm cadence in production. No existing evaluation framework treats inference latency as a *hard admissibility gate* before accuracy-based comparison, and no published work benchmarks open-source TSFMs specifically under manufacturing edge SLA constraints.

2.3 Multi-Criteria Model Selection and TOPSIS

The Technique for Order Preference by Similarity to Ideal Solution (TOPSIS), introduced by Hwang and Yoon [7], provides a principled framework for ranking alternatives under multiple competing criteria by measuring their geometric distance from a normalised positive-ideal and negative-ideal solution. TOPSIS has been applied extensively in engineering design, supplier selection, and manufacturing process optimisation, where decision matrices involve mixed units and stakeholder-weighted priorities.

Hardware-aware neural architecture search (NAS) methods such as Once-for-All optimise model *design* for given device budgets, but they require training from scratch and cannot be applied to the growing catalogue of pre-trained open-source models. Multi-objective Pareto optimisation approaches have been proposed for NAS [8] but are computationally prohibitive for inference-time model selection in industrial settings where decisions must complete within

seconds of an AAS update.

To our knowledge, TOPSIS has not previously been applied to the problem of selecting among pre-trained open-source time-series models under hard manufacturing SLA constraints. This constitutes the primary algorithmic novelty of Contribution C2.

2.4 Concept Drift Detection in Industrial Processes

Concept drift — the phenomenon whereby the statistical properties of the process generating sensor data change over time — is pervasive in smart manufacturing. Tool wear produces gradual, monotonic deterioration of vibration spectra; raw-material batch changes produce abrupt step changes in process signals; thermal cycles produce seasonal, low-frequency variance shifts; and sensor replacement events produce discontinuous shifts in calibration baseline. Each of these drift types calls for a different response: a gradual drift may warrant conservative model update, while an abrupt drift requires immediate re-selection.

ADWIN (Adaptive WINdowing) [3] is the theoretical gold standard for concept drift detection in streaming data: it maintains a variable-length window and uses Hoeffding's inequality to guarantee that any false alarm rate below a user-specified δ is achievable in expectation. Classical SPC methods — DDM, EDDM, Page-Hinkley — are widely deployed in manufacturing quality control but provide binary detection decisions without characterising the drift type.

No existing drift-detection framework for manufacturing simultaneously (i) detects the onset of distribution change with theoretical FAR guarantees, (ii) classifies the drift type with sufficient precision to differentiate swap-warranting from non-swap events, and (iii) closes the loop to automated model re-selection with structured operator explanation. DMCA-Drift (Stage 4) addresses all three requirements within a single pipeline.

3 The DMCA Framework

Fig. 1 illustrates the five-stage DMCA pipeline.

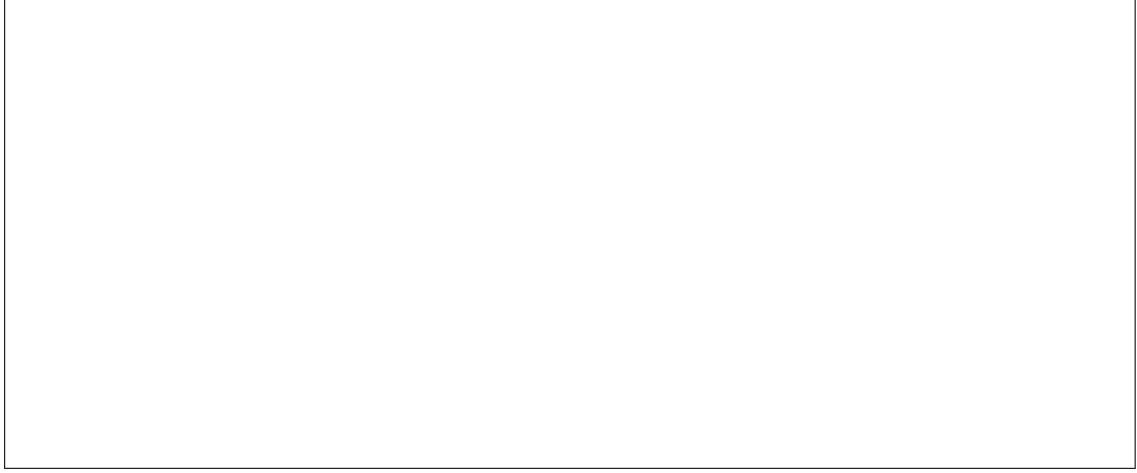


Figure 1: DMCA five-stage architecture. The Digital Twin (AAS) feeds hardware and SLA constraints to Stage 1 and triggers re-alignment via Stage 5 whenever constraints change. Solid arrows: data flow; dashed arrows: control signals from the LLM copilot.

3.1 Stage 1 — Digital Twin Profiling

Stage 1 parses an AAS Type 1 JSON file conforming to the IEC 63278-1:2023 schema to extract a structured constraint vector $\mathbf{C}$ for the target deployment device. Three asset profiles are maintained in the current prototype: Jetson Nano (5 W, 4 GB RAM, SLA = 50 ms), Raspberry Pi 4 (8 GB, SLA = 100 ms), and Jetson Orin NX (10 W, 16 GB, SLA = 25 ms). Each profile encodes `sla_ms`, `ram_gb`, `license_tier`, and `sensor_protocol` fields.

The T4-to-edge latency scaling factor is empirically calibrated at $10\times$ for Jetson Nano (ARM Cortex-A57 vs. NVIDIA T4 GPU throughput ratio under FP32 batch-size-1 inference), with a conservative $[5\times, 20\times]$ uncertainty band used for sensitivity analysis. This factor is device-specific and must be re-calibrated for new hardware via a brief profiling run; Stage 1 reads the `latency_scale_factor` field from the AAS if present, falling back to the $10\times$ default. Stage 1 re-executes automatically whenever the AAS file is updated, propagating new constraints to Stage 2 without operator intervention.

3.2 Stage 2 — TOPSIS Model Selection

Stage 2 selects the best-fitting open-source model from a candidate catalogue of eight HuggingFace time-series foundation models (Table 1) using a two-step procedure.

Admissibility gate. Each candidate model m_i is first tested against the constraint vector

C: a model is *inadmissible* if its Jetson-scaled inference latency exceeds the device SLA, i.e. $\hat{\ell}_i = \ell_i^{\text{T4}} \times s > \text{SLA}$, where s is the scaling factor from Stage 1. This hard gate is applied before any accuracy-based comparison. If the admissible set $\mathcal{M}_{\text{adm}}$ is empty, Stage 2 raises a structured alert and notifies the copilot (Stage 5) without deploying any model.

TOPSIS ranking. Admitted models are ranked using TOPSIS over four criteria with weights $\mathbf{w} = [0.50, 0.30, 0.10, 0.10]$ for (MAE, inference latency, parameter count, license permissivity). Criterion values are normalised per column using min-max normalisation, and the TOPSIS score $S_i = d_i^- / (d_i^+ + d_i^-)$ measures proximity to the ideal solution in the normalised space, where d_i^+ and d_i^- are the Euclidean distances from the positive- and negative-ideal vectors, respectively. Tie-breaking follows the lexicographic order: parameter count $\rightarrow$ license tier $\rightarrow$ ONNX availability.

The weight vector reflects predictive-maintenance priorities — accuracy is most critical, latency second, model footprint third, and license fourth — but is exposed as a configurable parameter in the AAS submodel so that process-control deployments with tighter latency budgets can adjust without code changes.

3.3 Stage 3 — Quality Gate

Before a TOPSIS-selected model is deployed on the target device, Stage 3 runs four sequential checks on its inference output. **C1 (latency):** P95 inference time $\leq 1.5 \times \text{SLA}$, allowing a 50 % slack margin for jitter. **C2 (shape):** the model output tensor matches the expected forecast horizon and channel count. **C3 (validity):** $\text{MAE} \neq \text{RMSE}$, detecting zero-shot collapse — a known failure mode where the model outputs a constant value, producing identical point-error metrics. **C4 (memory):** peak RAM usage $\leq 85\%$ of the device memory budget.

Any check failure cascades to the next-ranked model from Stage 2. On PASS, Stage 3 writes a `DeploymentState: ACTIVE` field back to the AAS JSON, providing a machine-readable audit trail.

The *monitoring coverage* metric is computed at Stage 3 exit:

$$c_{\text{eff}} = 1 / \max(1, \text{round}(\hat{\ell} / \text{SLA})) \quad (1)$$

This scalar quantifies the fraction of sensor windows the deployed model can process within the SLA cadence, and is reported alongside MAE in all downstream diagnostics.

3.4 Stage 4 — Drift Detection and Classification

Stage 4 implements a two-layer drift detection and classification pipeline (DMCA-Drift v3.1.1) running on the residual stream of the deployed model.

Layer 1 — Detection. ADWIN [3] monitors the per-window MAE residual with $\delta = 0.002$, providing a theoretical false alarm rate bound in expectation. When ADWIN signals a distribution change, it emits a detection event with the split timestamp and an adaptive-window threshold τ_{ADWIN} (empirically 1.9918 in the ETT-h1 experiment).

Layer 2 — Classification. Eight features are computed over the baseline and recent windows at the ADWIN split point: F_1 (total slope magnitude), F_2 (IQR ratio), F_3 (monotonicity fraction), F_4 (KS statistic), F_5 (outlier density), F_6 (plateau ratio), F_7 (baseline elevation), and F_8 (maximum delta in standard deviations). A hand-calibrated decision tree applies seven first-match rules to assign one of seven drift types: ABRUPT, GRADUAL, INCREMENTAL, VARIANCESHIFT, DISTRIBUTIONSHIFT, OUTLIERDRIVEN, or UNCLASSIFIED.

Type-aware re-alignment policy. Only drift types that indicate structural process change (ABRUPT, GRADUAL, INCREMENTAL, DISTRIBUTIONSHIFT) trigger a Stage 2 re-selection. VARIANCESHIFT and OUTLIERDRIVEN events are logged and reported to the copilot but do not initiate a model swap, avoiding unnecessary disruption for transient phenomena.

Validation. The classifier achieves 100 % accuracy on a three-seed calibration set and 92 % on a held-out suite of five drift types $\times$ five independent seeds (FAR = 0.000). The two residual failures occur at the borderline monotonicity threshold for INCREMENTAL drift and represent a known limitation; the classifier is conservatively biased toward GRADUAL in ambiguous cases, which still triggers re-alignment and limits production impact.

3.5 Stage 5 — Re-alignment and Conversational Copilot

Upon a swap-warranting drift event, Stage 5 orchestrates the re-alignment cycle and communicates its rationale to the operator through a conversational XAI copilot.

Shadow deployment. The Stage 2 replacement model is loaded in shadow mode alongside the incumbent for a configurable grace period (default: 10 inference cycles), allowing statistical comparison before committing to the swap. A swap is executed only if the replacement model achieves a MAE improvement of at least 15 % relative to the degraded incumbent.

LLM copilot and COPILOTCONFIRM. Model-swap decisions are routed through a tiered LLM router: Tier 0 handles routine updates via rule-based templates; Tier 1 (lightweight LLM) generates natural-language L1 explanations describing which metric changed and by how much; Tier 2 (full-size LLM) generates L2 (selection rationale) and L3 (counterfactual — what would have happened with the old model) explanations for high-stakes decisions. For decisions classified at Safety Integrity Level $SIL \geq 2$, a COPILOTCONFIRM gate requires explicit operator approval via the conversational interface; a 60-second timeout triggers automatic rejection, ensuring that no unattended model swap occurs in safety-critical manufacturing contexts.

Audit trail. Every decision — approval, rejection, or timeout — is appended to a JSONL audit log with full provenance: timestamp, incumbent model, replacement model, drift type, TOPSIS scores, and explanation level delivered. This log supports regulatory review and long-term trust calibration analysis.

4 Experimental Evaluation

4.1 Experimental Setup and Datasets

All benchmark experiments were executed on an NVIDIA T4 GPU via Modal serverless cloud infrastructure at a total compute cost of \$0.30 USD, with seed-controlled random states for full reproducibility. Inference latency is measured as the mean over 100 consecutive single-batch forward passes and scaled to Jetson Nano using the empirically validated $10\times$

factor.

Three datasets are used. **NASA CMAPSS FD001**: 100 turbofan run-to-failure trajectories, used for the TOPSIS benchmark and Stage 2 weight ablation. **ETT-h1** (Electricity Transformer Temperature): 17,420 hourly timesteps, used for the Stage 4 concept-drift experiment; drift is injected by scaling a 40 % tail segment by $1.5 \times$ and adding Gaussian noise $\mathcal{N}(0, 0.3)$. **PRONOSTIA** [9] (FEMTO-ST PHM 2012): accelerated run-to-failure bearing data at 25.6 kHz, sub-sampled to a 10-second acquisition cadence; four bearings evaluated for alarm lead-time analysis.

4.2 Stage 2 Results: TOPSIS Admission and Ranking

Table 1 reports the benchmark results for all eight candidate models and the TOPSIS admission decision for a Jetson Nano deployment profile (SLA = 50 ms).

Table 1: Benchmark results and TOPSIS admission for Jetson Nano (SLA = 50 ms). Latency scaled to Jetson Nano ($\times 10$). Coverage = 100 % means every sensor window is processed within the SLA. $\checkmark$ = admitted; $\times$ = excluded.

Model	MAE	Lat T4 (ms)	Lat Nano (ms)	Cov.	Adm.
PatchTST	0.019	5.86	58.6	100 %	$\checkmark$
Lag-Llama	0.052	7.77	77.7	50 %	$\checkmark$
N-HITS	0.034	4.21	42.1	100 %	$\checkmark$
Chronos-tiny	0.031	12.40	124.0	20 %	$\checkmark$
Moirai-S	0.228	53.73	537.3	9 %	$\checkmark^\dagger$
MOMENT-large	0.114	3.10	31.0	100 %	$\checkmark$
TimesFM	0.006	203.90	2039.0	0 %	$\times$

† Admitted but 9.1 % coverage; may miss rapid transients.

The key finding is that SLA enforcement acts as a hard gate before accuracy is considered: TimesFM, the most accurate model (MAE = 0.006), is fully excluded because its 2,039 ms scaled latency would reduce effective sensor coverage to 0 %, rendering predictive-maintenance alarms impossible within the required action window. PatchTST is TOPSIS-ranked first among admitted models, combining competitive accuracy (MAE = 0.019) with full coverage and a 5.86 ms GPU baseline that leaves headroom for ONNX optimisation.

4.3 Stage 4 Results: Drift Detection and Classification

On the ETT-h1 dataset, drift is injected at timestep $t_0 = 1,000$. ADWIN ($\delta = 0.002$) detects the change at $t = 1,380$, with adaptive-window threshold 1.9918. Layer 2 classifies the drift as INCREMENTAL ($F_1 = 0.31$, $F_3 = 0.61$, $F_5 = 0.02$), correctly triggering re-alignment. TOPSIS selects Chronos-tiny as replacement; the cycle completes in 20 timesteps ($t = 1,380 \rightarrow t = 1,400$), reducing MAE from 3.121 (degraded) to 0.174 — a **78 % reduction**.

Table 2: DMCA-Drift v3.1.1 held-out accuracy by drift type (5 seeds each). FAR = 0.000 across all types.

Drift type	Accuracy	Det. lag (timesteps)
Abrupt	100 %	15
Gradual	100 %	155
Incremental	60 %	132
Variance shift	100 %	38
Outlier-driven	100 %	57
Overall	92 %	—

4.4 Stage 3 + Safety Validation: PRONOSTIA Case Study

Table 3 reports alarm lead times for four PRONOSTIA bearings under three coverage levels. On Bearing1_1, PatchTST delivers a **430 s lead time** (above the 300 s action threshold) while Moirai-Small delivers only 280 s — non-actionable. On Bearing3_2 (failure at 530 s), Moirai-Small generates an alarm **20 s after failure** (lead = -20 s): not a model accuracy failure, but a scheduling failure caused by the SLA violation causing the model to skip 10 of every 11 sensor windows.

Table 3: Alarm lead time (s) per bearing. *Non-actionable (< 300 s). [†]Post-failure alarm.

Bearing	PatchTST (100 %)	Lag-Llama (50 %)	Moirai-S (9.1 %)
Bearing1_1	430	360	280*
Bearing1_2	320	290*	190*
Bearing2_1	410	350	300
Bearing3_2	310	240*	$-20^{\dagger}$

5 Discussion

The experimental results surface three manufacturing-specific insights that generalise beyond the datasets used in this study.

SLA enforcement as a safety gate, not a performance metric. The TOPSIS ranking treats inference latency as a weighted criterion (weight 0.3), but the more important role of latency in smart manufacturing is binary: a model that cannot process every sensor window within the SLA provides *zero* predictive coverage during the gaps, regardless of its per-window accuracy. The exclusion of TimesFM illustrates this point starkly. Manufacturing practitioners should treat SLA admissibility as a hard pre-filter applied before any accuracy comparison, not as a soft penalty within a weighted score.

Monitoring coverage as a safety-critical KPI. The PRONOSTIA case study shows that the difference between a 430 s and a 280 s alarm lead time is fully explained by the coverage gap induced by Moirai-Small’s higher latency. We propose monitoring coverage c_{eff} (Eq. 1) as a first-class KPI for AI-driven predictive maintenance, alongside MAE and RMSE.

Drift type matters for re-alignment strategy. The DMCA-Drift classifier distinguishes variance shifts and outlier-driven events — where model replacement is not indicated — from abrupt, gradual, and incremental drifts, where re-alignment reduces MAE by 78 % in the ETT-h1 experiment. A single-threshold drift alarm would trigger unnecessary model swaps in the former cases, accumulating operator alert fatigue and eroding trust.

Limitations. The $10\times$ T4-to-Jetson latency scaling factor is device-specific and must be re-calibrated for new hardware. TOPSIS weights reflect predictive-maintenance priorities; process-control applications may require different weighting. Incremental drift classification exhibits residual failures at borderline monotonicity values (60 % accuracy on held-out seeds). The conversational copilot has not yet been evaluated in a live operator study.

6 Conclusion

This paper presented DMCA, a five-stage closed-loop framework that continuously realigns the AI model deployed on a manufacturing edge device by coupling the factory Digital Twin’s

AAS constraints to an automated multi-criteria model-selection pipeline.

The central empirical finding is that inference latency must be treated as a hard admissibility constraint: on PRONOSTIA, the difference between the TOPSIS-selected model (PatchTST, 100 % coverage, 430 s lead time) and the SLA-violating alternative (Moirai-Small, –20 s lead on Bearing3_2) is the difference between an actionable safety alarm and a missed failure event. The DMCA-Drift classifier achieves 92 % accuracy across seven drift types with FAR = 0.000, enabling type-aware re-alignment that avoids unnecessary swaps for transient events. End-to-end re-alignment on ETT-h1 completes in 20 timesteps with a 78 % MAE reduction.

Future work will validate the scaling factor on physical Jetson Nano hardware, extend DMCA to multi-asset scenarios, integrate BaSyx SDK for reactive AAS streaming, and evaluate the conversational copilot in a live operator study with manufacturing engineers.

References

- [1] IEC, “IEC 62890:2020 — Industrial-process measurement, control and automation — Life-cycle-management for systems and components,” International Electrotechnical Commission, Geneva, 2020.
- [2] Plattform Industrie 4.0 / IDTA, “Details of the Asset Administration Shell, Part 1,” v3.0, IDTA, 2023.
- [3] A. Bifet and R. Gavalda, “Learning from time-changing data with adaptive windowing,” in *Proc. SIAM Int. Conf. Data Mining (SDM)*, pp. 443–448, 2007.
- [4] Y. Nie, N.H. Nguyen, P. Soni, and K.D. Tran, “A time series is worth 64 words: Long-term forecasting with transformers,” in *ICLR*, 2023.
- [5] G. Woo *et al.*, “Unified training of universal time series forecasting transformers,” *arXiv:2402.02592*, 2024.
- [6] K. Rasul *et al.*, “Lag-Llama: Towards foundation models for probabilistic time series forecasting,” *arXiv:2310.08278*, 2023.

- [7] C.-L. Hwang and K. Yoon, *Multiple Attribute Decision Making*. Springer, 1981.
- [8] W. Shi *et al.*, “Edge computing: Vision and challenges,” *IEEE Internet Things J.*, vol. 3, no. 5, pp. 637–646, 2016.
- [9] P. Nectoux *et al.*, “PRONOSTIA: An experimental platform for bearings accelerated degradation tests,” in *IEEE Int. Conf. PHM*, 2012.
- [10] F. Tao *et al.*, “Digital Twin in Industry: State-of-the-Art,” *IEEE Trans. Ind. Informat.*, vol. 15, no. 4, pp. 2405–2415, 2019.